Nexus HMS Technologies Used

Nexus HMS Enterprise Edition

Document control

- Product: Nexus HMS Enterprise Hotel Management System

- Release baseline: Phase 7B Enterprise Platform

- Document type: Technology and architecture guide

- Date: 27 August 2026

1. Architecture summary

Nexus HMS is a TypeScript pnpm workspace containing a Next.js web application, a NestJS REST API, shared database schema libraries, generated API support, and operational scripts. The backend is a modular monolith: each business area owns a controller, service, and module while the application is deployed as one API process.

```
Hotel workstation or mobile client
|
v
Next.js 15 web app ---- Socket.IO realtime connection
|
v
NestJS 10 API
|-- authentication, guards, modules, metrics
|-- EventEmitter2 event bus
|-- process-local cache
|-- Socket.IO Redis adapter
|-- Swagger / OpenAPI
| |
v v
PostgreSQL 16 Redis 7
through Drizzle ORM realtime backplane
```

2. Runtime stack

Frontend

- Next.js 15 App Router for routing, server rendering, and client pages.

- React 19 for component rendering and stateful UI.

- TypeScript for type safety.

- Tailwind CSS and ShadCN-style UI primitives for styling and common controls.

- TanStack Query for API data fetching and cache-aware client state.

- Recharts for BI and report visualizations.

- Socket.IO client for real-time subscriptions.

The frontend contains providers for authentication state, locale, and theme. Locale files support English, Amharic, Oromo, Arabic, and French. Arabic applies RTL document direction. Theme state persists in local storage and can apply tenant branding colors when configured.

Backend

- NestJS 10 for module composition, dependency injection, guards, controllers, and lifecycle.

- Express 5 adapter for HTTP handling.

- Passport and passport-jwt for bearer authentication.

- ‘@nestjs/swagger‘ and OpenAPI generation for API discovery.

- ‘@nestjs/throttler‘ for a default 300 requests-per-minute throttle.

- Helmet for HTTP security headers.

- Compression middleware for response compression.

- Socket.IO and NestJS WebSockets for real-time channels.

- ‘@socket.io/redis-adapter‘ and the ‘redis‘ client for cross-replica Socket.IO delivery.

Data layer

- PostgreSQL 16 for transactional relational storage.

- Drizzle ORM for typed SQL queries and schema definitions.

- Shared ‘lib/db‘ package for table definitions and database access.

- PostgreSQL ILIKE search for the current global search implementation.

- Local filesystem storage for uploaded documents in the current deployment boundary.

3. Security technologies

- bcryptjs password hash verification.

- JWT access and refresh tokens.

- A separate short-lived MFA-pending JWT secret derived from the session secret.

- Speakeasy for TOTP generation and verification.

- QRCode for authenticator enrollment QR data URIs.

- Node crypto AES-256-GCM for MFA secret and backup-code encryption at rest.

- Global fail-closed JWT guard and route-level guards.

- IP allowlist/blocklist support for IPv4 and IPv4 CIDR.

- NestJS ValidationPipe for request transformation and whitelisting behavior.

- Password-authenticated Redis connections, Kubernetes Secrets, and Redis NetworkPolicies.

‘SESSION_SECRET‘ is the critical runtime secret for token signing and the MFA encryption key derivation. The MFA module refuses to silently use a weak default when this secret is absent.

4. Phase 7B enterprise runtime

Event bus

‘@nestjs/event-emitter‘ and EventEmitter2 provide the process-local internal publish/subscribe interface. Reservation and room changes publish domain events consumed by the WebSocket gateway. Redis is not a durable EventBus and does not replace EventEmitter2.

Cache

The cache is an in-memory Map with expiry, statistics, prefix invalidation, and an explicit administrative flush. It remains process-local; the deployed Redis service is not used as a shared cache.

Real-time gateway

Socket.IO connections authenticate through the JWT handshake, subscribe to named channels, and receive tenant-scoped events. In production, the API uses Redis 7 through ‘@socket.io/redis-adapter‘ and the ‘redis‘ client so Socket.IO packets reach clients connected to other API replicas. ‘REDIS_URL‘ is required in production and must carry the Redis credential. The included Compose and Kubernetes Redis deployments require a matching password; the container smoke test authenticates its Redis readiness check and both API replicas. A connection counter is included in metrics.

Mobile

Mobile sessions store device identity, optional push token, platform, activity, and active state. Token refresh, delta synchronization, and batch push endpoints are included. Actual FCM/APNs delivery is outside this release.

Business intelligence

The BI module exposes KPI summaries, current-versus-prior comparisons, simple occupancy and revenue forecasts, and branch/category drill-downs. The frontend renders cards and Recharts visualizations.

Metrics

An interceptor tracks request totals and durations. The metrics controller returns Prometheus-compatible text. Metrics are public for standard scraping and should be restricted at the network boundary in production.

5. Workspace and build tooling

- pnpm workspace for package management.
- TypeScript 5.9 with project references and package-level type checking.
- Node.js 24 workspace runtime guidance, with container versions selected per release policy.
- Nest CLI for API development and compilation.
- Next CLI for web development and production builds.

- Prettier for consistent formatting.

- SQL migration files for database changes.

The database library must be compiled before the NestJS API requires it. Its runtime path points to compiled CommonJS output, which keeps the API build compatible with the shared package.

6. API contract and integration boundaries

The API uses a global ‘/api‘ prefix. JSON is the default payload format. Swagger is mounted at ‘/api/docs‘. Protected requests use:

```
Authorization: Bearer <access-token>
```

Redis 7 is a required production dependency for the implemented cross-replica Socket.IO adapter. The Kubernetes template stores ‘REDIS_PASSWORD‘ and the credential-bearing ‘REDIS_URL‘ in a Secret, permits Redis ingress only from API pods, and denies Redis egress. Redis and web pods explicitly run as non-root without privilege escalation. The included internal Redis service is password-protected but does not provide TLS; real production should prefer a managed, private TLS Redis service and a ‘rediss://‘ URL. Separate future integrations would still be required for a shared cache or a durable event bus. RabbitMQ or Kafka for durable event delivery, Elasticsearch for search, S3 or MinIO for object storage, and an external identity provider for OAuth2/OIDC/LDAP/SSO remain architectural extension points.

7. Architectural trade-offs

- A modular monolith keeps transactions and deployment simple while the hotel domain is still evolving.

- PostgreSQL and Drizzle provide a strong relational model and typed queries without adding a separate service layer.

- Redis-backed Socket.IO supports realtime delivery across API replicas, while cache entries and EventEmitter2 domain events remain process-local.

- The 10Gi ReadWriteOnce upload volume and required API pod affinity support the current three-replica, single-node storage boundary; multi-node API placement requires shared or object storage.

- Explicit public routes make Prometheus and refresh flows practical while global authentication remains fail-closed.

- Tenant-aware enterprise services are safer than pretending the legacy core tables already have complete tenant isolation.